

Data Imbalance in Machine Learning: Approaches for Dealing with Highly Imbalanced Datasets

ANDREAS LIZENBERGER, Department of Electrical Engineering and Computer Science, University of Siegen, Germany
CHRISTOPHER MAHLICH, Department of Electrical Engineering and Computer Science, University of Siegen, Germany

Abstract: Training machine learning models on imbalanced datasets is a difficult task. Learners tend to ignore and misclassify the minority class leading to poor performance. In the worst case, the learner only predicts the majority class. Data imbalance occurs frequently and naturally in real-world datasets. There are multiple techniques that help achieve good results despite working with highly imbalanced datasets. These techniques vary in their application and average effectiveness.

Additional Key Words and Phrases: data imbalance, imbalanced dataset, recall, specificity, precision, F-measure, ROC curve, undersampling, oversampling, SMOTE, cost-sensitive learning, one-class learning, bagging, boosting

ACM Reference Format:

Andreas Lizenberger and Christopher Mahlich. 2026. Data Imbalance in Machine Learning: Approaches for Dealing with Highly Imbalanced Datasets. 1, 1 (April 2026), 6 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 INTRODUCTION

Binary classification is a common task in a machine learning setting [3]. A learner, e.g. a neural network, must predict for each sample in a given dataset whether the sample is in class A or in class B, e.g. is the patient healthy or is the patient sick? In supervised learning, each sample in the dataset is labeled with the class that the sample belongs to. The learner is trained on a portion of the data (training data) and then evaluated on an unseen portion of the data (test set). The learner creates a prediction for each sample in the test set and that prediction is compared against the label.

Data Imbalance occurs when the ratio between class A, the minority class, and class B, the majority class, is heavily skewed towards the majority class [10]. In the typical scenario, accuracy is a good metric to evaluate the performance of the learner but fails for an imbalanced dataset. Accuracy cannot detect learners that only predict the majority class. We discuss alternative metrics like recall, specificity, precision, ROC curve and F1-score and show that these metrics are meaningful in an imbalanced setting.

Learners generalize better when trained with datasets that are balanced. When trained on an imbalanced dataset learners tend to misclassify samples of the minority class [8]. We discuss approaches that deal with imbalanced datasets to improve the performance

Authors' addresses: Andreas Lizenberger, andreas.lizenberger@student.uni-siegen.de, Department of Electrical Engineering and Computer Science, University of Siegen, Hölderlinstraße 3, Siegen, Germany, 57076; Christopher Mahlich, christopher.mahlich@student.uni-siegen.de, Department of Electrical Engineering and Computer Science, University of Siegen, Hölderlinstraße 3, Siegen, Germany, 57076.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM XXXX-XXXX/2026/4-ART

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

of the learners. There are data-level approaches (undersampling, oversampling, SMOTE), algorithm-level approaches (cost-sensitive learning, one-class learning) and ensemble approaches (bagging, boosting). We discuss the advantages and disadvantages of the approaches and find that ensemble approaches are on average performing better than the other presented approaches.

Multi-class classification uses more than two labels. The aforementioned concepts and approaches are presented for binary classification only and serve to provide an overview and keep the survey succinct. In the literature, there are equivalent definitions for multi-class settings.

2 PERFORMANCE METRICS

Performance metrics indicate the performance of a learner. As an example let us look at a dataset containing patients and their condition. We set the label to 1 (positive label) if the patient is ill and we set the label to 0 (negative label) if the patient is healthy. In an imbalanced setting ill patients are the minority class and healthy patients are the majority class. The given learner provides a prediction for each patient. The prediction is a number between 0 and 1 and represents the probability of being in either class. Rounding the prediction by setting a threshold of 0.5 provides a concrete prediction (positive or negative prediction).

2.1 Confusion Matrix

We can compare the concrete predictions against the labels to see how well the learner performs. There are 4 cases to consider:

- True Negative (TN): prediction = 0, label = 0
- False Negative (FN): prediction = 0, label = 1
- False Positive (FP): prediction = 1, label = 0
- True Positive (TP): prediction = 1, label = 1

We can count how many of the samples fall into each case and display the numbers as seen in Table 1.

Table 1. Confusion Matrix.

		Prediction	
		Positive	Negative
Label	Positive	TP	FN
	Negative	FP	TN

This is called confusion matrix and provides absolute numbers [9]. The confusion matrix is useful to define the performance metrics that give relative numbers.

2.2 Accuracy

Accuracy is frequently used when it comes to evaluating learners. It measures how many predictions are made correctly [9]. In our case,

we compute how many ill patients are correctly classified as ill and how many healthy patients are correctly classified as healthy. We compute and divide the sum by the total number of patients.

$$\text{Accuracy} = \frac{\text{TN} + \text{TP}}{\text{TN} + \text{FN} + \text{FP} + \text{TP}}$$

What are the problems with accuracy? Let us assume that the learner only predicts the majority class and that the dataset has an imbalance ratio of 1:1000. We obtain the confusion matrix from Table 2. The resulting accuracy is 99.9%. This is a misrepresentation of

Table 2. The learner only makes negative predictions.

		Prediction	
		Positive	Negative
Label	Positive	0	1
	Negative	0	999

the performance of the learner. The learner is useless but accuracy is unable to detect this problem [9]. We must consider the false negatives and false positives as well.

2.3 Recall, Specificity and Precision

Recall measures how many of the ill patients were classified as ill and specificity measures how many of the healthy patients were classified as healthy. Precision computes how many of the patients that were classified by the learner as ill are actually ill [9].

$$\text{Recall} = \frac{\text{TP}}{\text{FN} + \text{TP}} \quad \text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

$$\text{Precision} = \frac{\text{TP}}{\text{FP} + \text{TP}}$$

Recall is also known as sensitivity or true positive rate. Specificity is also referred to as true negative rate [9].

2.4 Receiver Operating Characteristic (ROC) Curve

The ROC curve is a graph that shows how well the recall of the learner performs against the false positive rate (1– specificity) of the learner. The y-axis represents recall and the x-axis the FP rate. We assume that the learner provides a probability of the sample being in the minority group. A typical threshold to create a binary classification is 0.5, however, this threshold can be set arbitrarily. Samples that are rated below that threshold are classified as part of the majority class and samples above that threshold are classified as part of the minority class. A point on the ROC curve represents the performance of the learner for a given threshold. Setting the threshold lower than 0.5 favors the minority class. This increases the number of TP and FP. The opposite effect is achieved when the threshold is set high [7].

ROC has multiple advantages: 1. The threshold can be optimized by neglecting the FP rate. For example, a higher number of people are classified as ill and must stay home but the number of people that are indeed ill and correctly classified also increases. This can be a valid trade-off if the illness is highly contagious. 2. Thresholds in the ROC curve that have similar recall are easy to compare. Pick the threshold with a lower FP rate. 3. Algorithms can be easily

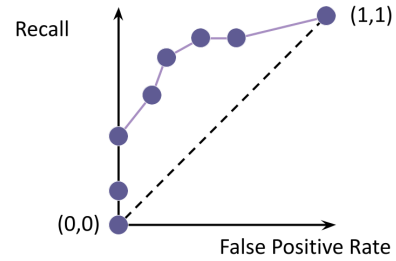


Fig. 1. Example ROC curve.

compared with AUC that is the area under the ROC curve. A bigger AUC means the algorithm is overall better at separating samples in the minority and majority class [7].

2.5 F1-score

F1-score, or F-measure, combines precision and recall to evaluate the performance of a model in a more balanced way. F1-score is defined as the harmonic mean of precision and recall and can be calculated as follows [9]:

$$\text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Normally a higher F1-score also means a better overall performance of a classification model. The score is a number between 0 and 1 where 1 stands for optimal performance and 0 for poor performance. It should be noted that in some cases precision may be more important than recall and vice versa. The interpretation of the F1-score depends on the particular use case and goal of the classification model. F1-score is particularly useful when the class distribution is imbalanced because F1-score includes both false positive and false negative cases. When the performance of the model is measured with F1-score then the minority class must have positive labels. In contrast, defining the minority class as negative can lead to incorrect results because the performance of the model is evaluated concerning the majority class. F1-score can also be used for multi-class problems. For this, we calculate the F1-score per class. This means that we evaluate the success of each class separately as if there were separate classifications for each class. Takahashi et al. described in their work 2 techniques to calculate a single F1-score for the whole classifier.

One is the micro-average F1-score, which summarises all classification results across all classes. The second is the macro-average F1-score, which is a simple average of the F1-scores calculated for each class [16].

3 DECISION BOUNDARIES

Generally, learners attempt to construct decision boundaries that separate the majority class and minority class within the feature space that models the data. Let us investigate why data imbalance causes problems with our learners [1]:

1. When the minority class is underrepresented then the resulting decision boundaries can be simply too small. This happens when crucial samples that represent the true distribution are missing in

the provided dataset. The learner is unable to estimate the true decision boundary because the data is simply not available.

2. When the minority class and majority class are not distinct in their features then class overlapping can occur. In Fig. 2 in the right graph we can see that all minority samples fit into a single decision boundary for the underlying distribution. However, in the left graph is the decision boundary that has been constructed for the imbalanced dataset. The samples of the minority class do not form a proper cluster. Some samples of the minority class end up being interpreted as noise within the majority class.

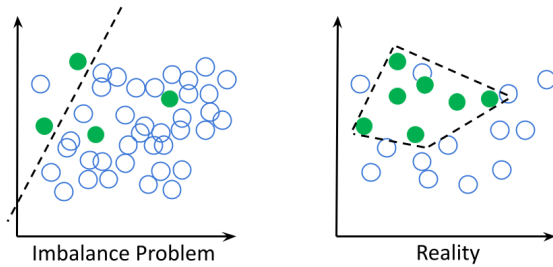


Fig. 2. Minority class appears as noise.

3. When the minority class consists of multiple individual clusters that themselves are imbalanced with regards to each other then this problem is called within class imbalance. In this case, the previous two problems might apply to the individual clusters. It can become difficult for the learner to find decision boundaries that fit all the subclasses.

4 DATA-LEVEL APPROACH

Data-level approaches use the original dataset to create a new dataset with a more balanced ratio. Let n be the number of minority samples and m be the number of majority samples. There are two approaches to consider:

1. The ratio $r = n/m$ between the majority and minority class can be improved by adding samples to the minority class. We can achieve the new ratio $r' = (n + n')/m$ by computing $n' = \lceil r' \cdot m - n \rceil$. The data-level algorithms create or select n' -many minority samples.
2. We create or select m' -many samples of the majority class and keep n fixed. The data-level algorithms run for $m' = \lceil n/r' \rceil$ iterations.

4.1 Random Oversampling

Random oversampling (ROS) is a technique used in machine learning to balance the class distribution. The basic idea is to randomly select data from the minority class and duplicate it multiple times to normalize the ratio between the majority class and the minority class. The disadvantage of this approach is that the model is more susceptible to overfitting because the dataset contains many duplicates of the same sample. Another disadvantage is that the generation of additional samples of the minority class can double

the size of the dataset. This can lead to longer training times and increase the resources required for training the model, especially for large datasets. Oversampling is typically used when the dataset is small or when there is limited potential to collect new samples [6].

Mohammed et al. show that oversampling generally performs well on most classifiers and achieves high scores for different evaluation metrics [12][6].

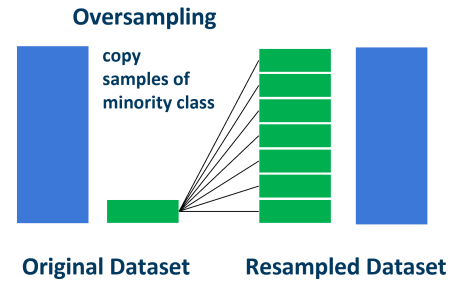


Fig. 3. Oversampling duplicates minority samples [12].

4.2 Random Undersampling

Random undersampling (RUS) is an alternative to ROS when it comes to balancing the class distribution. Samples are removed from the majority class in order to normalize the ratio between the majority class and the minority class. This approach speeds up the training process because the size of the dataset is reduced. A major flaw is that important data can be lost. Typically undersampling is used when there is a significant difference between the size of data points in the majority and minority class [6].

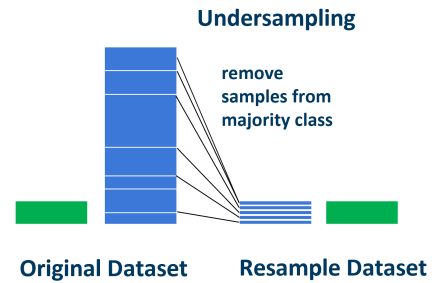


Fig. 4. Undersampling removes majority samples [12].

4.3 SMOTE

SMOTE stands for synthetic minority oversampling technique and aims to generate new samples of the minority class to rebalance the distribution in the dataset. The approach only considers the subset of all minority samples and ignores the subset of all majority samples. The algorithm can be outlined in the following steps [4]:

- (1) Randomly select a sample d from the minority subset.
- (2) Randomly select a k -nearest neighbor d_k of d .
- (3) Randomly select a line ratio $\alpha \in]0, 1[$.
- (4) Compute a synthetic minority sample $m = d + \alpha(d_k - d)$.
- (5) Add m to the list of synthetic minority samples.

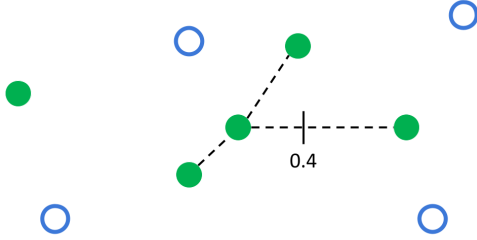


Fig. 5. SMOTE interpolates nearby minority samples.

SMOTE has some practical limitations and cannot be used for any dataset. Imagine a dataset of human faces, interpolating two faces does not create a proper face e.g. two noses, too many eyes, etc. Additionally, there must be some definition of distance between the samples in the dataset to compute the k -nearest neighbor. This distance must reflect the similarity of two samples otherwise SMOTE creates synthetic samples that do not make sense semantically.

A dataset of words cannot be SMOTEd because words cannot be interpolated. We require some vector space that reflects the intensities of the features.

By construction, all synthetic samples lie on lines between two minority samples. This can introduce noise in the dataset. Let us assume that the majority samples are clustered around $(0, 0)$ and there are two minority samples at $(-1, 0)$ and $(1, 0)$. SMOTE fits an arbitrary line that goes through the cluster in the center.

Oversampling approaches are prone to overfitting. An excessive overuse of SMOTE can cause the learner to overfit to those aforementioned lines.

5 ALGORITHM-LEVEL APPROACH

The goal is to modify existing learners to reduce the bias towards the majority class by adjusting weights during training or using special loss functions for imbalanced data.

5.1 Cost-sensitive learning

The basic idea of cost-sensitive learning is that each class is assigned a different cost for misclassification [11]. These costs are mapped in a cost matrix (see Fig. 6).

A specific value c_{ij} in the matrix that is shown in Fig. 6 represents the cost of predicting class i when the actual class is j . If $i = j$ the costs are set to 0. In other words, the true positives and true negatives have no costs. The costs for the false negative and false positive can be adjusted depending on the dataset and the specific problem. The costs can be determined by experts, by using machine learning models or by trial and error. In the specific case that we have a highly imbalanced dataset the costs for the minority class are set higher than the costs for the majority class.

	Positive Prediction	Negative Prediction
Positive Label	$C(1/1)$	$C(1/0)$
Negative Label	$C(0/1)$	$C(0/0)$

Fig. 6. Cost matrix.

Ling et al. choose between two methods: direct and meta-learning methods to use cost-sensitive learning [11]. The direct method modifies the main algorithm, while the meta-learning method is used to transform existing cost-insensitive classifiers into cost-sensitive classifiers without modifying the actual models. This is done by a middleware component that preprocesses training data or post-processes the output of the cost-insensitive learning algorithms. There is also cost-sensitive meta-learning that is a hybrid approach.

An example of a direct method is the application of cost-sensitive learning to logistic regression where the error function is modified by adding the costs to the error. This penalizes the model more for incorrectly predicting the minority class and the model pays more attention to the minority class.

5.2 One-class learning

The basic idea of the one-class classifier is that the model is only trained on the data of the majority class, also known as the positive class. The model focuses on learning the positive class and is therefore well characterized, while the negative class has no training instances. The goal is to create a normality of the positive training data in order to define a decision boundary that includes all training data. This decision boundary allows us to classify new data points by checking if they fall within the boundary and therefore belong to the positive class. If they fall outside the boundary they may belong to the negative or minority class [18].

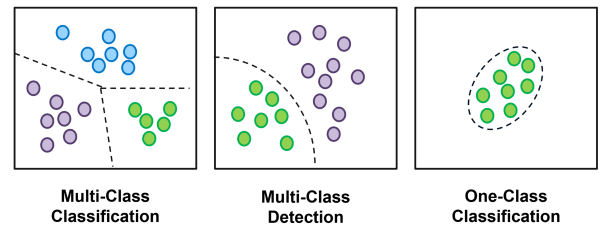


Fig. 7. Different forms of classification [13].

Fig. 7 shows the difference between learning from one class and learning from multiple classes. Both multi-class classification and detection contain data from different classes during training. In

one-class classification, there is only one class to learn from for the classifier.

One of the common algorithms that can be used to create one-class learning classifiers is the one-class support vector machine (OCSVM). It is based on the same concepts as the traditional support vector machine (SVM) but has a different goal. The traditional SVM aims to divide the input data into two or more classes, while the OCSVM aims to create a minimum volume hypersphere that encloses the data of the positive class. The goal is to find a decision boundary or threshold that captures the normal data points while rejecting those that are too far from the normal points. In this way, OCSVM can detect anomalies or objects of interest in the data with high accuracy [17].

The one-class learning approach is often used in specific cases where it is difficult to get representative data for all classes, especially in anomaly detection scenarios where the “normal” class is well-defined but the “anomalous” classes are difficult to define.

6 ENSEMBLE APPROACH

Approaches that combine modifications to the datasets and training algorithms fall under the category of hybrid methods. A promising subcategory of hybrid methods is ensemble approaches that create multiple weak learners. Each learner delivers a decent performance on a subset of the original data. The strength of ensemble approaches comes from training multiple weak learners and combining each individual prediction of the learners to a final prediction [10].

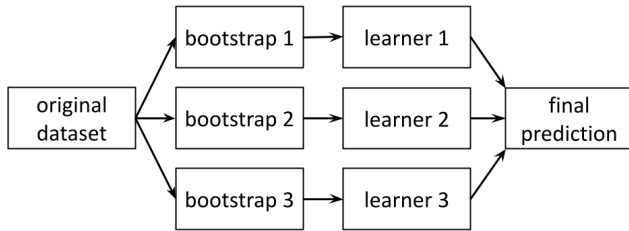


Fig. 8. General outline of ensemble learning.

A bootstrap of the dataset L is any dataset L_k of the same size as L and L_k is a possible result of randomly sampling over L [5].

6.1 Bagging

Bagging is an acronym for bootstrap aggregating and is an ensemble technique to create and train multiple learners. Each learner is trained on a bootstrap and has an equal vote on the final prediction. A dataset can be viewed as a distribution that follows an unknown real-world distribution. If multiple learners are trained on such datasets then their average prediction is a better approximation of the unknown real-world distribution. The creation of bootstraps is a technique that attempts to replace and imitate those datasets. The algorithm operates in the following way [2]:

- (1) Create bootstrap L_t of the original dataset L .
- (2) Train learner f_t on L_t .
- (3) For T different learners compute the final prediction:

$$f(d) = \frac{1}{k} \sum_{t=1}^T f_t(d)$$

If the learner stores many parameters then creating, training and storing each learner can be expensive. However, bagging has the advantage that each step can be performed in parallel.

Bagging does not work for every learner. If the learner is resistant against bootstrapping, e.g. k -nearest neighbor, then bagging has minimal effect [14].

6.2 Boosting

Boosting is an ensemble technique to create and train multiple learners. Each learner is trained based on the performance of the previous learner. This is achieved by creating bootstraps that contain samples that the previous learners have misclassified frequently. After a short number of iterations, the bootstraps become more balanced than the original dataset. The final prediction is a weighted sum of the individual predictions of the learners. Compared to bagging each learner is well known and receives a performance coefficient that rates the overall performance on the original dataset. As an example, we outline AdaBoost with a binary learner [15].

Let L be the original dataset. A weighted dataset consists of samples $d \in L$ and their corresponding weights w and labels y . The first weighted dataset $L_0 = L$ has the starting weights $w = 1/|L|$ for all samples. AdaBoost with a binary learner generates a new learner f_t and updates the weights of L_0 :

- (1) Use the weights of L_{t-1} as a distribution to bootstrap L_t .
- (2) Train f_t on L_t .
- (3) Compute the error e as the number of samples that were incorrectly classified.
- (4) Compute the performance coefficient α_t for f_t :

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1-e}{e} \right)$$

- (5) Update the weights:

$$w = w \exp(-\alpha_t y f_t(d))$$

- (6) Normalize the weights to achieve a distribution.

The outlined algorithm can be run T times to generate the learners $f_1, f_2, \dots, f_T$. The final prediction is a weighted sum over all predictions of the learners and their performance coefficients:

$$f(d) = \text{sgn} \left(\sum_{t=1}^T \alpha_t f_t(d) \right)$$

If a learner f_t performed poorly on a given sample d then the corresponding weight w is increased, if the learner performed well on the sample the weight is decreased. In the next iteration the bootstrap L_{t+1} is more likely to contain samples that the previous learner has misclassified.

The disadvantage of boosting is that training is done sequentially. Just like with bagging every learner must be trained and stored and this is expensive.

Boosting creates learners that perform better on samples that the previous learners misclassified. On the one hand, this produces very good final predictions. On the other hand, this can cause overfitting to the training data [15].

Noisy data can be a problem for boosting. The bootstraps can become polluted with outliers and noise. This can impede the generalization process of the learners. However, boosting is good at detecting those outlier, which then can be filtered [15].

7 PRACTICAL RESULTS

Rodriguez et al. compare the effectiveness of different approaches for dealing with data imbalance concerning software defect prediction. Software defect prediction attempts to predict the probability that software components or modules contain bugs or defects. Shepperd et al. have bundled and preprocessed multiple datasets of projects from NASA into the versions MDP and PROMISE. Both versions have been preprocessed again by removing duplicates and errors. This results in versions MDP D', MDP D'', PROMISE D' and PROMISE D''. The ratio between minority class and majority class ranges between 1% and 30%. The datasets are labeled for binary classification. Rodriguez et al. use Matthew's Correlation Coefficient, ROC and F1-score to evaluate the following techniques: RUS, ROS, SMOTE, AdaBoost.M1, Bagging and others [14].

For the given setting RUS and ROS have performed quite well. A ratio (minority:majority) of 25:75 performs better than a ratio of 50:50. RUS and ROS achieve better results for the preprocessed versions of MDP and PROMISE. Ensemble methods on average rank higher than sampling or cost-sensitive methods.

8 CONCLUSION

We have discussed the problems of highly imbalanced datasets. We have seen that a significant bias towards the majority class can strongly influence the ability of the learner to construct good decision boundaries. Accuracy is not a good metric to evaluate the learner in an imbalanced setting. Recall, specificity, precision, ROC curve and F1-score are more suitable for that task. We have investigated different approaches to deal with imbalanced datasets. RUS and ROS are simple tools that work fairly well. Ensemble approaches deliver on average even better results.

REFERENCES

- [1] Aida Ali, Siti Mariyam Shamsuddin, and Anca I. Ralescu. Classification with class imbalance problem. *Int. J. Advance Soft Compu. Appl*, 5(3):176–204, 2013.
- [2] Leo Breiman. Bagging predictors. *Machine learning*, 24:123–140, 1996.
- [3] Gürol Canbek, Tugba Taskaya Temizel, and Seref Sagiroglu. PToPI: A comprehensive review, analysis, and knowledge representation of binary classification performance measures/metrics. *SN Computer Science*, 4(1):13, 2022. ISSN 2661-8907. doi: 10.1007/s42979-022-01409-1. URL <https://doi.org/10.1007/s42979-022-01409-1>.
- [4] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. Smote: Synthetic minority over-sampling technique. *J. Artif. Int. Res.*, 16(1): 321–357, jun 2002. ISSN 1076-9757.
- [5] Bradley Efron and Robert J Tibshirani. *An introduction to the bootstrap*. CRC press, 1994.
- [6] Vaishali Ganganwar. An overview of classification algorithms for imbalanced datasets. *International Journal of Emerging Technology and Advanced Engineering*, 2(4):42–47, 2012.
- [7] Zhe Hui Hoo, Jane Candlish, and Dawn Teare. What is an roc curve?, 2017.
- [8] Nathalie Japkowicz et al. Learning from imbalanced data sets: a comparison of various strategies. In *AAAI workshop on learning from imbalanced data sets*, volume 68, pages 10–15. Citeseer, 2000.
- [9] Justin M. Johnson and Taghi M. Khoshgoftaar. Survey on deep learning with class imbalance. *Journal of Big Data*, 6(1):27, 2019. ISSN 2196-1115. doi: 10.1186/s40537-019-0192-5. URL <https://doi.org/10.1186/s40537-019-0192-5>.
- [10] Joffrey L. Leevy, Taghi M. Khoshgoftaar, Richard A. Bauder, and Naeem Seliya. A survey on addressing high-class imbalance in big data. *Journal of Big Data*, 5:42, 11 2018. ISSN 2196-1115. doi: 10.1186/s40537-018-0151-6. URL <https://doi.org/10.1186/s40537-018-0151-6>.
- [11] Charles X. Ling and Victor S. Sheng. *Cost-Sensitive Learning*, pages 231–235. Springer US, Boston, MA, 2010. ISBN 978-0-387-30164-8. doi: 10.1007/978-0-387-30164-8_181. URL https://doi.org/10.1007/978-0-387-30164-8_181.
- [12] Roweida Mohammed, Jumanah Rawashdeh, and Malak Abdullah. Machine learning with oversampling and undersampling techniques: Overview study and experimental results. In *2020 11th International Conference on Information and Communication Systems (ICICS)*, pages 243–248, 2020. doi: 10.1109/ICICS49469.2020.239556.
- [13] Pramuditha Perera, Poojan Oza, and Vishal M. Patel. One-class classification: A survey. *ArXiv*, abs/2101.03064, 2021. URL <https://api.semanticscholar.org/CorpusID:231418911>.
- [14] Daniel Rodriguez, Israel Herraiz, Rachel Harrison, Javier Dolado, and José C. Riquelme. Preliminary comparison of techniques for dealing with imbalance in software defect prediction. In *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering, EASE '14*, New York, NY, USA, 2014. Association for Computing Machinery. ISBN 9781450324762. doi: 10.1145/2601248.2601294. URL <https://doi.org/10.1145/2601248.2601294>.
- [15] Robert E. Schapire. *The Boosting Approach to Machine Learning: An Overview*, pages 149–171. Springer New York, New York, NY, 2003. ISBN 978-0-387-21579-2. doi: 10.1007/978-0-387-21579-2_9. URL https://doi.org/10.1007/978-0-387-21579-2_9.
- [16] Kanae Takahashi, Kouji Yamamoto, Aya Kuchiba, and Tatsuki Koyama. Confidence interval for micro-averaged f1 and macro-averaged f1 scores. *Applied Intelligence*, 52(5):4961–4972, 2022. ISSN 1573-7497. doi: 10.1007/s10489-021-02635-5. URL <https://doi.org/10.1007/s10489-021-02635-5>.
- [17] David Tax and Robert Duin. Support vector data description. *Machine Learning*, 54:45–66, 01 2004. doi: 10.1023/B:MACH.0000008084.60811.49.
- [18] Chih-Fong Tsai and Wei-Chao Lin. Feature selection and ensemble learning techniques in one-class classifiers: An empirical study of two-class imbalanced datasets. *IEEE Access*, 9:13717–13726, 2021. doi: 10.1109/ACCESS.2021.3051969.